

TESTLEAF GEN AI TRAINING



Prompt Engineering

A structured journey from basics to advanced techniques

Phase 1: Why & How

Phase 2: Quality

Phase 3: ICEPOT

Phase 4: Controls

Phase 5: Advanced

Your Prompt Engineering Journey

PHASE 1

Why & How

What is a prompt · How prompts work

PHASE 2

Quality Intuition

Weak vs strong · Bad → Better → Best

PHASE 3

ICEPOT Framework

Golden framework · Persona · Output formats · Evaluation

PHASE 4

Precision Controls

Keywords · Caps · Temperature control

PHASE 5

Advanced Techniques

Zero/One/Few-shot · CoT · Step-back · Socratic · Iterative

PHASE 1

Why & How

Understanding what prompts are and how AI processes them

Slides 4 – 5

What Is a Prompt?

A prompt is any input you give to an AI model to guide its response.



Instruction

Tells the AI what to do
"Generate a test case for login"

Context

Background the AI needs
"This is a banking app with 2FA"



Input Data

The content to work on
"Here is the API spec: ..."



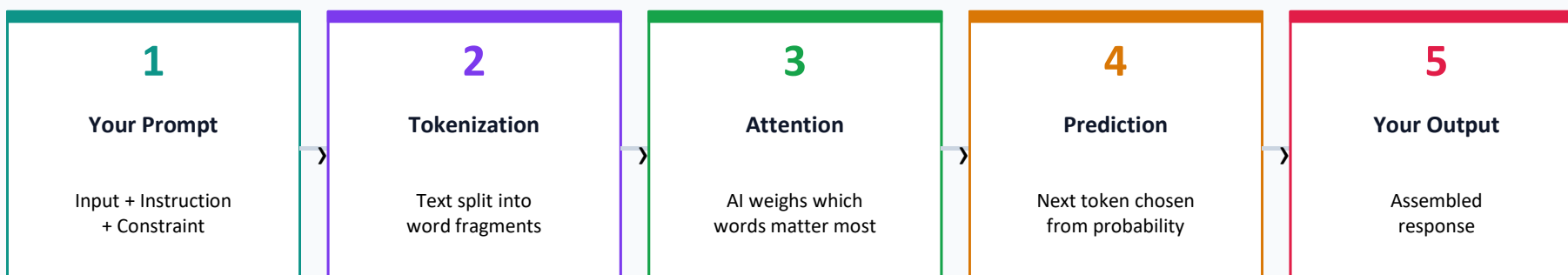
Output Signal

Format you expect back
"Respond in Gherkin BDD
format"

💡 **Key insight:** The quality of your output is determined entirely by the quality of your input.

How Prompts Work for AI

AI doesn't 'understand' like humans — it predicts the most likely next token based on your prompt.



Example:

Prompt: "Translate this English sentence to clean Spanish. Ensure a poetic tone."

→ New Input: sentence to translate | Instruction: translate | Constraint: poetic tone

PHASE 2

Quality Intuition

See the difference between weak and strong prompts

Slides 7 – 8

Weak Prompt vs Strong Prompt

The same task — generating a Selenium test script — yields completely different results based on prompt quality.

✗ WEAK PROMPT

```
"Generate a Selenium test script."
```

- ✗ No context about language
- ✗ No target application
- ✗ No format specified
- ✗ No test scenarios defined
- ✗ Output: generic, unusable

✓ STRONG PROMPT

```
"You are a test automation assistant.  
Generate a Selenium Java test script to  
verify login on a Salesforce app.  
Include setup/teardown. Use explicit waits."
```

- ✓ Clear role + persona
- ✓ Target app + language specified
- ✓ Specific requirements listed
- ✓ Output: structured, production-ready

Bad → Better → Best in QA

Prompting in QA: the difference between average and exceptional output is your prompt.

✗ BAD

"Write test cases for login page"

No context

No details

No format

No edge cases

Generic, shallow, not usable

⚡ BETTER

"Write test cases for a login page with username and password fields. Include positive and negative scenarios."

Some context ✓

Pos/neg included ✓

No business rules

No format specified

Decent but incomplete

★ BEST

"Act as Senior QA Engineer. Write detailed test cases for login with: username (email format), password (8+ chars, uppercase, number, special). Include positive, negative, edge, security. Format: Test ID | Scenario | Steps | Expected | Priority"

Clear role ✓

Detailed requirements ✓

Security + edge cases ✓

Defined output format ✓

Detailed, structured, production-ready

PHASE 3

The ICEPOT Framework

The golden framework for writing consistently great prompts

Slides 10 – 13

The ICEPOT Golden Framework

Six components that transform any vague prompt into a structured, high-quality instruction.

I**Instructions**

What the AI must do — the core task

P**Persona**

Role the AI should play (→ see next slide)

C**Context**

Background, system, environment info

O**Output Format**

Structure, length, format of the response (→ TLDR/ELI5)

E**Examples**

Sample inputs/outputs to guide behaviour

T**Tone**

Formal, casual, technical, empathetic...

O – Output Format: TLDR, ELI5 & ELI15

The 'O' in ICEPOT. Controlling output length and complexity level changes everything about how useful the response is.

TL;DR

Too Long; Didn't Read

Executive summary. Ask for this when you want the essential point in 1–3 sentences. Ideal for reports, articles, or long outputs.

"Give me a TL;DR of this document."

ELI5

Explain Like I'm 5

Simplest possible explanation using analogies and everyday language. Perfect for introducing unfamiliar concepts to any audience.

"ELI5: what is overfitting in ML?"

ELI15

Explain Like I'm 15

Technical but approachable. Assumes some education but avoids deep jargon. Great for intermediate audiences and business stakeholders.

"ELI15: how does attention work in a transformer?"

💡 **Pro tip:** You can chain these — "Give me a TL;DR first, then an ELI5 explanation."

PHASE 5

Advanced Techniques

Techniques that go beyond instructions — teach the AI to reason

Slides 17 – 21

Prompting Techniques: Zero, One & Few-Shot

How many examples you provide dramatically changes the quality and style of the AI's output.

Zero-Shot

No examples given.
AI relies entirely on
the instruction alone.

```
"Classify this review as positive or  
negative: \"The app crashed  
twice.\""
```

→ Negative

Use when: Quick tasks, simple classification, code gen
with clear spec

One-Shot

One example given.
AI learns the pattern
from a single sample.

```
"Review: Great app! → Positive  
Review: Crashes often. → ?"
```

Use when: When zero-shot gives inconsistent format
or style

Few-Shot

2–5 examples given.
AI picks up nuance,
tone, and edge cases.

```
"Great app! → Positive  
Crashes! → Negative  
Meh, ok → Neutral  
This feature broke my flow. → ?"
```

Use when: Complex tasks, tone matching, domain-
specific outputs

CoT — Chain-of-Thought Prompting

Instead of asking for an answer directly, ask the AI to show its reasoning step by step.

✗ Without CoT

"If a test suite has 120 tests and 15% fail, how many pass?"

→ AI might answer: "102" — with no reasoning shown.
If wrong, you can't tell where it failed.

Problem: No visibility into reasoning. Errors are invisible.

✓ With CoT

"...Let's think step by step."

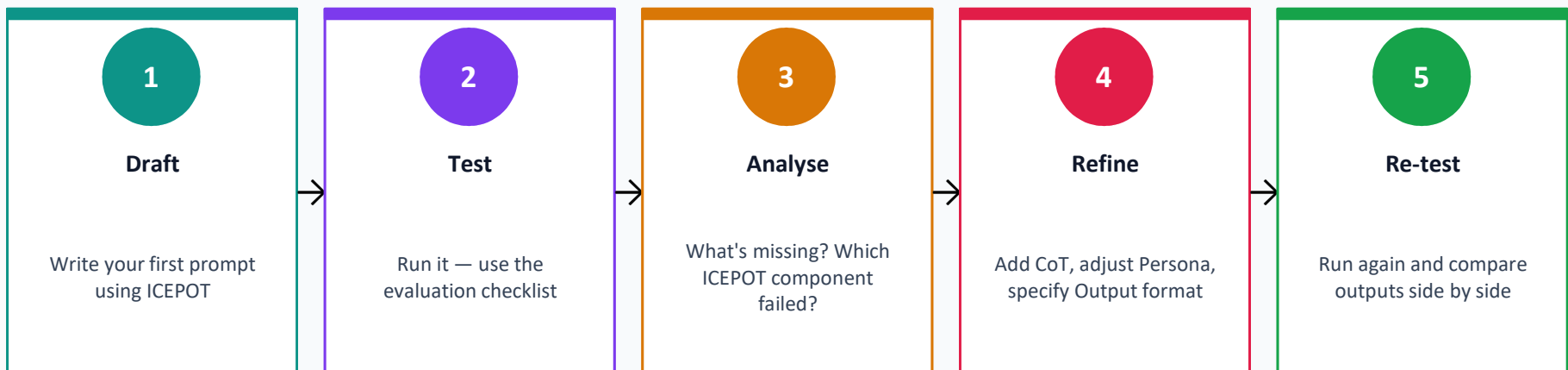
→ Step 1: 15% of 120 = 18 fail
→ Step 2: 120 - 18 = 102 pass
→ Answer: 102

Benefit: Reasoning is visible, auditable, and correctable.

✧ Magic phrases: "Let's think step by step." · "Walk me through your reasoning." · "Show your work."

Iterative Prompt Development — The Capstone Skill

Every technique you've learned feeds into this: the discipline of refining prompts through structured cycles.



↺ Repeat until output quality matches your standard

✓ You now have the full toolkit: ICEPOT → Keywords → Zero/One/Few-Shot → CoT → Step-Back → Socratic → Iterate

SUMMARY



Your Prompt Engineering Toolkit

Phase 1

Know what prompts are and how AI tokenises and weighs them

Phase 2

Spot the difference between weak and strong prompts instantly

Phase 3

Apply ICEPOT · Role vs Instruction · Output formats · Evaluate with the checklist

Phase 4

Control creativity with keywords and CAPS to enforce hard constraints

Phase 5

Use Zero/One/Few-Shot → CoT → Step-Back → Socratic → Iterate to mastery

Better Prompts. Better Output. Better QA.